

Python for Data Science

Buổi 1: Data Science Foundation, Handbook Overview, Environment Setup, Python OOP Foundation

PGS-TS Nguyễn Mạnh Hùng

Ngày 17 tháng 8 năm 2026

Cấu trúc Buổi 1

- 1 Module 0: Data Science Foundation
- 2 Module 1: Python Data Science Handbook Overview
- 3 Module 2: Python Data Science Environment Setup
- 4 Module 3: Python OOP Foundation định hướng Data Science

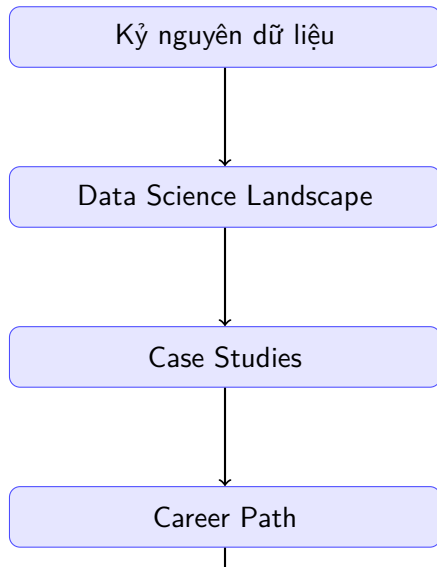
Lưu ý phạm vi

Buổi học tập trung vào nền tảng Data Science, Python ecosystem, môi trường thực hành và OOP trong Python. Buổi học chưa đi sâu vào công cụ Big Data như Hadoop, Spark, Kafka, Flink, Hive hay Data Lakehouse.

Chuẩn đầu ra (Learning Outcomes)

Sau khi hoàn thành module này, học viên có khả năng:

- 1 Trình bày khái niệm Data Science và bối cảnh ra đời trong kỷ nguyên dữ liệu.
- 2 Phân biệt Data Science với Artificial Intelligence và Machine Learning.
- 3 Phân tích ứng dụng thực tế của Data Science thông qua các case study điển hình.
- 4 Mô tả các vai trò nghề nghiệp chính trong lĩnh vực dữ liệu.
- 5 Trình bày quy trình (workflow) thực hiện một dự án Data Science.



Chúng ta đang sống trong kỷ nguyên dữ liệu

Mỗi ngày chúng ta tạo dữ liệu từ:

- Điện thoại thông minh, mạng xã hội, email.
- Website, thương mại điện tử, thanh toán điện tử.
- GPS, camera, đồng hồ thông minh, cảm biến IoT.
- Hệ thống doanh nghiệp như CRM, ERP, Sales, Customer Service.

Thông điệp

Mỗi hành động số đều có thể tạo ra dữ liệu. Vấn đề không chỉ là có dữ liệu, mà là biết cách biến dữ liệu thành hiểu biết hữu ích.

Một ngày bình thường tạo ra dữ liệu như thế nào?



Các hành động trên tạo ra dữ liệu người dùng, dữ liệu vị trí, dữ liệu giao dịch và dữ liệu hành vi.

Dữ liệu đến từ đâu?

Người dùng

- Facebook
- TikTok
- YouTube
- Search

Doanh nghiệp

- CRM
- ERP
- Sales
- Support

Thiết bị

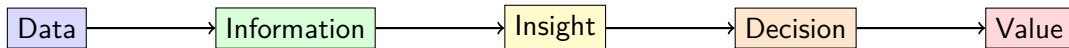
- GPS
- Camera
- Smartwatch
- IoT Sensor

KB $\rightarrow$ MB $\rightarrow$ GB $\rightarrow$ TB $\rightarrow$ PB $\rightarrow$ EB $\rightarrow$ ZB

- Dữ liệu tăng nhanh theo quy mô toàn cầu: IDC ước tính lượng dữ liệu toàn cầu đạt khoảng 175 ZB vào năm 2025.
- Con người không thể phân tích thủ công lượng dữ liệu lớn như vậy.
- Cần công cụ, quy trình và tư duy mới để khai thác dữ liệu.

Nguồn: Reinsel, Gantz & Rydning (2018), The Digitization of the World: From Edge to Core, IDC White Paper.

Từ Data đến Business Value



Ví dụ: Netflix phân tích lịch sử xem phim để đề xuất nội dung phù hợp, từ đó tăng trải nghiệm và giữ chân người dùng.

Big Data là gì?

Big Data thường được mô tả bằng mô hình 3V do Laney (2001) đề xuất:

Volume

Khối lượng dữ liệu rất lớn.

Velocity

Dữ liệu sinh ra liên tục.

Variety

Nhiều dạng dữ liệu: bảng, văn bản, hình ảnh, âm thanh, video.

Các mô hình sau này mở rộng thành 5V, bổ sung thêm **Veracity** (độ tin cậy của dữ liệu) và **Value** (giá trị khai thác được từ dữ liệu).

Big Data có phải là Data Science?

Big Data / Data Engineering

Hạ tầng dữ liệu, lưu trữ phân tán, xử lý quy mô lớn. Ví dụ: Hadoop, Spark, Kafka, Data Lake.

Data Science

Phân tích, trực quan hóa, thống kê, mô hình dự đoán, ra quyết định dựa trên dữ liệu.

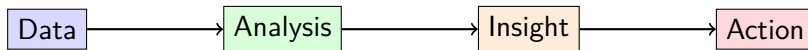
Phạm vi khóa học

Khóa học giới thiệu các kỹ thuật xử lý và phân tích dữ liệu bằng Python, chưa giới thiệu sâu các công cụ Big Data.

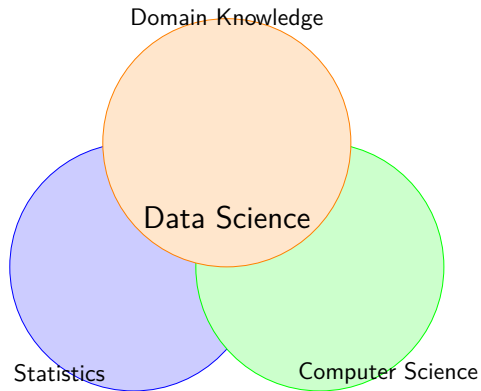
Data Science là gì?

Định nghĩa

Data Science là quá trình sử dụng dữ liệu để tạo ra tri thức, hỗ trợ ra quyết định và giải quyết các bài toán thực tế, kết hợp phương pháp thống kê, thuật toán tính toán và tri thức chuyên ngành (Dhar, 2013).



Data Science nằm ở đâu?



Data Science nằm ở giao điểm giữa năng lực thống kê/toán học, kỹ năng lập trình và tri thức chuyên ngành (Conway, 2010).

Data Science vs Data Analytics

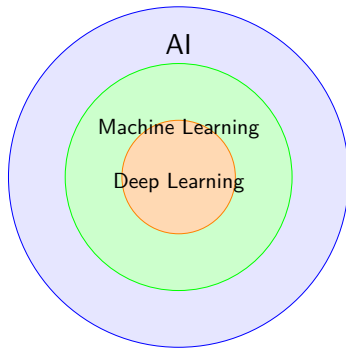
Data Analytics

- Điều gì đã xảy ra?
- Dashboard
- KPI
- Reporting

Data Science

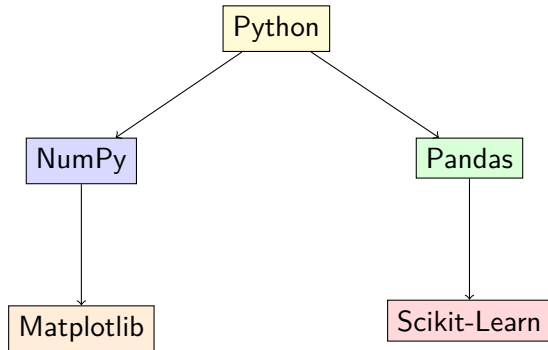
- Điều gì sẽ xảy ra?
- Prediction
- Machine Learning
- Optimization

AI vs Machine Learning vs Deep Learning



- **AI:** lĩnh vực rộng nghiên cứu hệ thống có khả năng thực hiện tác vụ đòi hỏi trí tuệ con người.
- **Machine Learning:** nhánh của AI, hệ thống học quy luật từ dữ liệu thay vì được lập trình tường minh.
- **Deep Learning:** nhánh của Machine Learning dựa trên mạng nơ-ron nhiều lớp (Goodfellow, Bengio & Courville, 2016).

Python trong hệ sinh thái Data Science



Tại sao cần Case Study?

Data Science không chỉ là code hoặc thuật toán. Điều quan trọng là hiểu cách Data Science giải quyết bài toán thực tế trong doanh nghiệp.

- 1 Netflix Recommendation
- 2 Uber Demand Forecasting
- 3 Fraud Detection
- 4 Amazon Recommendation
- 5 Healthcare Analytics

Case Study: Netflix Recommendation

Bài toán

Làm thế nào để hiển thị đúng bộ phim mà người dùng có khả năng muốn xem?

Dữ liệu có thể bao gồm lịch sử xem, đánh giá, tìm kiếm, thời lượng xem, phim bỏ dở.

User Activity → Watch History → Recommendation Engine → Suggested Movies

Nguồn: Gomez-Uribe & Hunt (2016), *The Netflix Recommender System*, ACM TMIS.

Case Study: Uber Demand Forecasting

Bài toán

Trong khoảng thời gian tiếp theo, khu vực nào sẽ có nhu cầu gọi xe cao nhất?

Dữ liệu có thể gồm GPS, lịch sử đặt xe, thời tiết, sự kiện, giao thông và mật độ dân cư.

Historical Data → Forecast Model → Demand Prediction → Driver Allocation

Minh họa dựa trên các báo cáo kỹ thuật công khai của Uber Engineering về bài toán dự báo nhu cầu (demand forecasting) theo chuỗi thời gian quy mô lớn.

Case Study: Fraud Detection

Ngân hàng xử lý rất nhiều giao dịch. Cần phát hiện giao dịch bất thường.

- Giá trị giao dịch
- Địa điểm và thời điểm giao dịch
- Thiết bị sử dụng
- Lịch sử hành vi khách hàng

Transactions → ML Model → Fraud Alert

Nguồn: Bolton & Hand (2002), *Statistical Fraud Detection: A Review*, Statistical Science.

Case Study: Amazon Recommendation

Bài toán

Làm thế nào để gợi ý sản phẩm phù hợp với hàng triệu người dùng và hàng triệu sản phẩm theo thời gian thực?

Amazon giới thiệu kỹ thuật **item-to-item collaborative filtering**: thay vì so sánh người dùng với người dùng, hệ thống so sánh sản phẩm với sản phẩm dựa trên lịch sử mua hàng chung.

Purchase History → Item Similarity Matrix → “Khách hàng đã mua sản phẩm này cũng mua...”

Nguồn: Linden, Smith & York (2003), *Amazon.com Recommendations: Item-to-Item Collaborative Filtering*, IEEE Internet Computing.

Case Study: Healthcare Analytics

Bài toán

Bệnh nhân nào có nguy cơ tái nhập viện hoặc diễn biến xấu trong thời gian tới?

Dữ liệu có thể gồm hồ sơ bệnh án điện tử (EHR), kết quả xét nghiệm, chỉ số sinh hiệu và tiền sử bệnh.

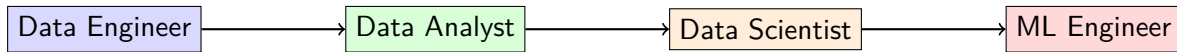
Electronic Health Records → Predictive Model → Risk Score → Clinical Decision Support

Nguồn: Obermeyer & Emanuel (2016), *Predicting the Future — Big Data, Machine Learning, and Clinical Medicine*, New England Journal of Medicine.

Tổng kết Case Studies

Lĩnh vực	Bài toán	Kỹ thuật gợi ý
Streaming	Recommendation	Prediction
Transportation	Demand Forecasting	Time Series
Banking	Fraud Detection	Classification
E-commerce	Recommendation	Personalization
Healthcare	Risk Prediction	Analytics

Data Career Landscape



Tổng quan các vai trò

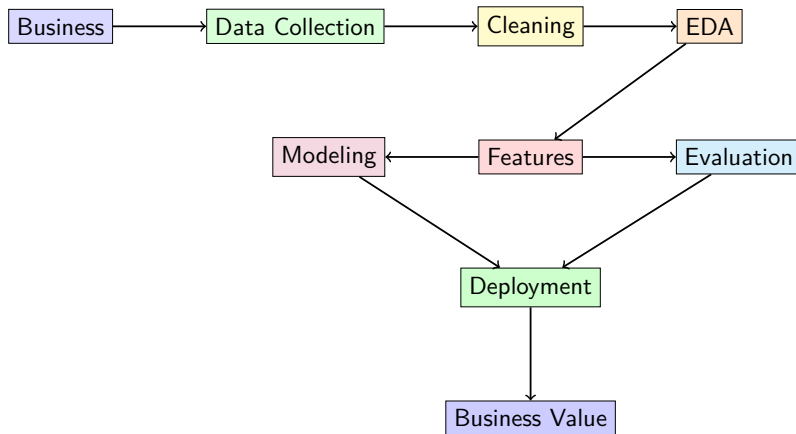
Vai trò	Trọng tâm	Mục tiêu
Data Analyst	Reporting	Hiểu hiện tại
Data Scientist	Prediction	Dự đoán tương lai
ML Engineer	Deployment	Đưa AI vào sản phẩm
Data Engineer	Infrastructure	Xây nền tảng dữ liệu

Data Scientist từng được Harvard Business Review mô tả là “công việc gọi cảm nhất thế kỷ 21” (Davenport & Patil, 2012).

Phạm vi khóa học

Tập trung chủ yếu vào nền tảng cho Data Analyst và Data Scientist; chưa đi sâu Data Engineering.

Data Science Workflow



Quy trình trên là một biến thể rút gọn của chuẩn CRISP-DM (Chapman et al., 2000), một trong những mô hình quy trình phổ biến nhất cho các dự án khai phá dữ liệu và Data Science.

Câu hỏi đầu tiên không phải là thuật toán nào nên dùng, mà là doanh nghiệp đang muốn giải quyết vấn đề gì.

Ví dụ

Khách hàng nào có khả năng rời bỏ dịch vụ trong 30 ngày tới?

Business Problem → Data Science Question → Machine Learning Problem.

Data Collection

Mục tiêu là thu thập đúng, đủ và liên quan tới bài toán.

Internal

- CRM
- ERP
- Sales

External

- Weather
- Open Data
- Social Media

Realtime

- GPS
- IoT
- Clickstream

- Data Cleaning: xử lý missing value, duplicate, sai định dạng, outlier.
- EDA: hiểu phân bố, xu hướng, outlier, mối quan hệ giữa các biến.
- Feature Engineering: biến dữ liệu thô thành biến đầu vào hữu ích cho mô hình.

Birth Date $\rightarrow$ Age ; Timestamp $\rightarrow$ Hour

Modeling, Evaluation, Deployment

- Modeling: Regression, Classification, Clustering.
- Evaluation: Accuracy, Precision, Recall, F1, MAE, RMSE, R^2 .
- Deployment: API, Website, Mobile App, Recommendation Engine.

Thông điệp

Machine Learning chỉ là một bước trong toàn bộ Data Science Workflow.

Module 1: Python Data Science Handbook Overview

Mục tiêu

Hiểu cấu trúc cuốn Python Data Science Handbook, vai trò từng chương và cách các chương liên kết với workflow Data Science.

Tại sao chọn Python Data Science Handbook?

Về cuốn sách

Python Data Science Handbook (VanderPlas, 2016), NXB O'Reilly Media, là tài liệu tham khảo chuẩn cho hệ sinh thái công cụ Data Science trong Python.

- Tập trung vào công cụ nền tảng của Data Science bằng Python.
- Nội dung có thể học qua Jupyter Notebook.
- Đi từ môi trường làm việc tới xử lý dữ liệu, trực quan hóa và Machine Learning.

Cấu trúc tổng thể Handbook

Chương	Nội dung chính
1	IPython và Jupyter Notebook
2	NumPy
3	Pandas
4	Matplotlib
5	Machine Learning với Scikit-Learn

Mỗi thư viện gắn với một công trình học thuật nền tảng: NumPy (Harris et al., 2020), Pandas (McKinney, 2010), Matplotlib (Hunter, 2007), Scikit-Learn (Pedregosa et al., 2011).

Workflow	Công cụ	Chương
Làm việc tương tác	Jupyter	1
Tính toán số	NumPy	2
Làm sạch dữ liệu	Pandas	3
EDA	Pandas + Matplotlib	3–4
Mô hình hóa	Scikit-Learn	5

Kỹ năng đạt được sau Handbook

- Làm việc với Jupyter Notebook.
- Xử lý dữ liệu số với NumPy.
- Làm sạch và biến đổi dữ liệu với Pandas.
- Tạo biểu đồ phân tích với Matplotlib.
- Xây dựng mô hình ML cơ bản với Scikit-Learn.

Module 2: Environment Setup

Mục tiêu

Học viên có thể cài đặt, tạo môi trường ảo, quản lý package và dùng VS Code/Jupyter Notebook để thực hành Data Science.

Tại sao cần Environment Setup?

Lỗi trong Data Science không chỉ đến từ code. Lỗi có thể đến từ sai phiên bản Python, thiếu package, xung đột thư viện, notebook dùng nhầm môi trường hoặc máy mỗi người có cấu hình khác nhau.

Khái niệm

Virtual Environment là môi trường Python độc lập cho từng project. Mỗi môi trường có interpreter và package riêng, theo chuẩn được mô tả trong PEP 405 (Python Enhancement Proposal).

Máy tính $\rightarrow$ Project A Env $\rightarrow$ Python + Packages A

Máy tính $\rightarrow$ Project B Env $\rightarrow$ Python + Packages B

Tạo môi trường Conda

```
conda create -n ds_course python=3.11
conda activate ds_course
conda install numpy pandas matplotlib scikit-learn notebook
python --version
conda list
jupyter notebook
```

VS Code và Python Extension

- VS Code là editor.
- Python Extension thêm hỗ trợ Python: chạy code, IntelliSense, debugging, chọn interpreter.
- Python Interpreter là chương trình thực sự chạy code Python.

Kiểm tra interpreter trong VS Code

Tạo file `check_env.py`:

```
import sys
print(sys.version)
print(sys.executable)
```

Nếu đường dẫn thuộc environment đang học, VS Code đã chọn đúng interpreter.

Jupyter Notebook là tài liệu tương tác kết hợp code, văn bản giải thích, dữ liệu, công thức và hình ảnh trực quan.

- Code Cell
- Markdown Cell
- Output
- Visualization

Magic Commands

```
%timeit  
%matplotlib inline  
%run script.py  
%pwd  
%ls
```

- Không cài package học tập trực tiếp vào base.
- Mỗi project có environment riêng.
- Xuất `environment.yml` khi cần chia sẻ.
- Luôn kiểm tra interpreter trước khi chạy code.
- Notebook tốt không chỉ chạy được, mà còn phải đọc được.

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Phân biệt built-in class, user-defined class, attribute và method.
- 2 Áp dụng encapsulation (property), class method và static method.
- 3 Sử dụng magic methods để tùy biến hành vi object (`__repr__`, `__len__`, `__getitem__`, `__call__`).
- 4 Áp dụng inheritance, abstract base class và polymorphism trong thiết kế mô hình.
- 5 Giải thích vì sao Pandas, Scikit-Learn, PyTorch và OpenMMLab được thiết kế theo phong cách hướng đối tượng.

Tại sao Data Science cần OOP?

Trong Data Science và Deep Learning, học viên thường gặp:

- `df.head()`, `df.describe()`
- `model.fit(X, y)`, `model.predict(X)`
- `fig, ax = plt.subplots()`, `ax.plot(x, y)`
- `class Net(nn.Module): ...` trong PyTorch
- `@MODELS.register_module()` trong OpenMMLab

Tất cả các câu lệnh này đều liên quan tới object, method, attribute và class – hiểu OOP là điều kiện tiên quyết để đọc hiểu và tùy biến mã nguồn của các thư viện Data Science và Deep Learning hiện đại.

Mọi thứ trong Python đều là Object

```
x = 10
name = "Python"
scores = [8, 9, 10]
print(type(x))
print(type(name))
print(type(scores))
```

Kết quả: int, str, list đều là class. Ngay cả `numpy.ndarray` và `torch.Tensor` cũng là class – `type(torch.tensor([1,2,3]))` trả về `torch.Tensor`.

Built-in Class: list

```
features = ["age", "income", "monthly_fee"]  
features.append("support_tickets")  
features.sort()  
print(features)
```

Ứng dụng: lưu danh sách feature, tên cột, điểm đánh giá model, danh sách batch loss trong một epoch huấn luyện.

Built-in Class: tuple

```
dataset_shape = (1000, 12)
rows, columns = dataset_shape
print(rows)
print(columns)
```

```
image_shape = (3, 224, 224) # (channels, height, width)
```

Tuple phù hợp cho dữ liệu cố định như kích thước dữ liệu, `tensor.shape` trong PyTorch, tọa độ hoặc cặp giá trị.

Built-in Class: dict

```
customer = {  
    "customer_id": "C001",  
    "age": 32,  
    "monthly_fee": 15.5,  
    "churn": False  
}  
print(customer["customer_id"])  
print(customer.get("age"))  
  
model_cfg = {"type": "ResNet", "depth": 50, "num_classes": 10}
```

Dictionary biểu diễn một record dữ liệu. Trong OpenMMLab, toàn bộ kiến trúc mô hình cũng được mô tả bằng một dict gọi là *config*.

Built-in Class: set

```
labels = ["cat", "dog", "cat", "bird", "dog", "cat"]
unique_labels = set(labels)
print(unique_labels)
print(len(unique_labels))
num_classes = len(set(labels))
```

Ứng dụng: xác định số lớp (`num_classes`) duy nhất từ tập nhãn trước khi xây dựng mô hình classification.

Built-in Class: str

```
review = "This Movie Was Absolutely Great!"  
tokens = review.lower().replace("!", "").split()  
print(tokens)  
vocab = sorted(set(tokens))
```

Ứng dụng: bước tiền xử lý văn bản (lower-casing, tokenize thô, xây vocabulary) trước khi đưa vào mô hình NLP/Deep Learning.

Tạo class Customer

```
class Customer:
    def __init__(self, customer_id, age, monthly_fee, support_tickets):
        self.customer_id = customer_id
        self.age = age
        self.monthly_fee = monthly_fee
        self.support_tickets = support_tickets

    def show_info(self):
        print(self.customer_id, self.age, self.monthly_fee)

c1 = Customer("C001", 32, 15.5, 2)
c1.show_info()
```

Method tính rủi ro churn

```
class Customer:
    def __init__(self, customer_id, age, monthly_fee, support_tickets):
        self.customer_id = customer_id
        self.age = age
        self.monthly_fee = monthly_fee
        self.support_tickets = support_tickets

    def churn_risk(self):
        if self.support_tickets >= 3 or self.monthly_fee > 30:
            return "High"
        return "Low"
```

Class Attribute và Instance Attribute

```
class Experiment:
    total_runs = 0 # class attribute: dùng chung cho mọi instance

    def __init__(self, name, learning_rate):
        self.name = name # instance attribute
        self.learning_rate = learning_rate
        Experiment.total_runs += 1

exp1 = Experiment("baseline", 0.01)
exp2 = Experiment("baseline-aug", 0.001)
print(Experiment.total_runs) # 2
```

Class attribute phù hợp để đếm số lượng experiment/model đã tạo trong một pipeline huấn luyện.

Class Dataset đơn giản

```
class Dataset:
    def __init__(self, records):
        self.records = records

    def summary(self):
        n = len(self.records)
        churned = sum(1 for r in self.records if r["churn"])
        print(f"So mau: {n}, so churn: {churned}")

data = Dataset([
    {"customer_id": "C001", "churn": True},
    {"customer_id": "C002", "churn": False},
])
data.summary()
```

Đây là bước đệm để xây dựng khái niệm Dataset object – nền tảng của `torch.utils.data.Dataset` sẽ trình bày ở phần sau.

Vì sao cần Encapsulation?

Encapsulation là nguyên tắc che giấu dữ liệu nội bộ của object, chỉ cho phép truy cập thông qua method/property được kiểm soát.

- `name`: attribute công khai (public).
- `_name`: quy ước “protected”, không nên truy cập trực tiếp từ bên ngoài.
- `__name`: “private”, Python thực hiện name mangling để hạn chế truy cập.

Vì sao quan trọng trong Deep Learning?

Encapsulation giúp bảo vệ trọng số mô hình (`__weights`) khỏi bị gán sai giá trị từ bên ngoài, tránh lỗi khó debug khi huấn luyện.

Property: Validate tham số huấn luyện

```
class TrainingConfig:
    def __init__(self, learning_rate):
        self._learning_rate = learning_rate

    @property
    def learning_rate(self):
        return self._learning_rate

    @learning_rate.setter
    def learning_rate(self, value):
        if value <= 0:
            raise ValueError("learning_rate phải > 0")
        self._learning_rate = value

cfg = TrainingConfig(0.01)
cfg.learning_rate = 0.001    # hợp lệ
cfg.learning_rate = -1       # raise ValueError
```

Property: Thuộc tính chỉ đọc (read-only)

```
class Dataset:
    def __init__(self, records):
        self._records = records

    @property
    def num_samples(self):
        return len(self._records)

data = Dataset([{"id": 1}, {"id": 2}, {"id": 3}])
print(data.num_samples)  # 3
data.num_samples = 10    # raise AttributeError: không thể gán
```

`num_samples` luôn được tính lại từ `_records`, không thể bị gán sai giá trị – tránh lỗi mất đồng bộ dữ liệu.

@classmethod: Alternative Constructor

```
import csv

class Dataset:
    def __init__(self, records):
        self.records = records

    @classmethod
    def from_csv(cls, path):
        with open(path) as f:
            records = list(csv.DictReader(f))
        return cls(records)

data = Dataset.from_csv("customers.csv")
```

Cùng ý tưởng với `pd.DataFrame.from_dict()` hay `Model.from_pretrained()` trong các thư viện Deep Learning: tạo object bằng nhiều cách khác nhau mà không cần nhiều tham số trong `__init__`.

@staticmethod: Hàm tiện ích gắn với class

```
class MetricUtils:
    @staticmethod
    def accuracy(y_true, y_pred):
        correct = sum(int(a == b) for a, b in zip(y_true, y_pred))
        return correct / len(y_true)

acc = MetricUtils.accuracy([1, 0, 1, 1], [1, 0, 0, 1])
print(acc)  # 0.75
```

staticmethod không cần self hay cls: dùng khi hàm liên quan về mặt logic tới class nhưng không cần truy cập instance.

Magic Method: `__str__` và `__repr__`

```
class Customer:
    def __init__(self, customer_id, age):
        self.customer_id = customer_id
        self.age = age

    def __str__(self):
        return f"Khach hang {self.customer_id}, {self.age} tuoi"

    def __repr__(self):
        return f"Customer(customer_id={self.customer_id!r}, age={self.age})"

c1 = Customer("C001", 32)
print(c1)      # gọi __str__: hiện thị cho người dùng
c1             # trong console/log gọi __repr__: phục vụ debug
```

`__repr__` nên trả về chuỗi rõ ràng, hữu ích khi debug model hoặc in log huấn luyện.

Operator Overloading: Vector2D

```
class Vector2D:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector2D(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f"({self.x}, {self.y})"

v1 = Vector2D(1, 2)
v2 = Vector2D(3, 4)
print(v1 + v2)
```

Đây chính là nguyên lý giúp `torch.Tensor` hỗ trợ cú pháp `a + b`, `a * b`: PyTorch overload `__add__`, `__mul__` cho lớp `Tensor`.

__len__ và __getitem__: Dataset kiểu PyTorch

```
class ImageDataset:
    def __init__(self, image_paths, labels):
        self.image_paths = image_paths
        self.labels = labels

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        return self.image_paths[idx], self.labels[idx]

ds = ImageDataset(["cat1.jpg", "dog1.jpg"], [0, 1])
print(len(ds))          # gọi __len__
print(ds[0])            # gọi __getitem__ -> ("cat1.jpg", 0)
for path, label in ds:
    print(path, label)
```

Đây chính xác là giao diện bắt buộc của `torch.utils.data.Dataset` trong PyTorch.

__call__: Object gọi được như hàm

```
class LinearModel:
    def __init__(self, w, b):
        self.w = w
        self.b = b

    def __call__(self, x):
        return self.w * x + self.b
```

```
model = LinearModel(w=2.0, b=1.0)
y = model(3.0)    # gọi model như một hàm, thực chất gọi __call__
```

Trong PyTorch, `model(x)` thực chất gọi `nn.Module.__call__`, và `__call__` sẽ tự động gọi lại `forward(x)` mà ta định nghĩa.

Inheritance và super()

```
class Employee:
    def __init__(self, name):
        self.name = name

class DataScientist(Employee):
    def __init__(self, name, specialty):
        super().__init__(name)
        self.specialty = specialty

ds = DataScientist("An", "Machine Learning")
```

Method Overriding

```
class Employee:
    def work(self):
        print("Employee is working")

class DataScientist(Employee):
    def work(self):
        print("Building ML models")

DataScientist().work()
```

Abstract Base Class với module abc

```
from abc import ABC, abstractmethod

class BaseScaler(ABC):
    @abstractmethod
    def fit(self, values):
        ...
    @abstractmethod
    def transform(self, values):
        ...

class MinMaxScaler(BaseScaler):
    def fit(self, values):
        self.min_, self.max_ = min(values), max(values)
        return self
    def transform(self, values):
        return [(x - self.min_) / (self.max_ - self.min_) for x in values]
```

BaseScaler không thể khởi tạo trực tiếp (BaseScaler() sẽ raise TypeError) – abc buộc lớp con phải hiện thực đầy đủ method trừu tượng.

Abstract Base Class: thêm một cách chuẩn hóa khác

```
class StandardScaler(BaseScaler):  
    def fit(self, values):  
        n = len(values)  
        self.mean_ = sum(values) / n  
        var = sum((x - self.mean_) ** 2 for x in values) / n  
        self.std_ = var ** 0.5  
        return self  
  
    def transform(self, values):  
        return [(x - self.mean_) / self.std_ for x in values]  
  
scaler = StandardScaler()  
scaler.fit([10, 20, 30, 40])  
print(scaler.transform([10, 20, 30, 40]))
```

MinMaxScaler và StandardScaler cùng kế thừa BaseScaler, cùng giao diện fit/transform nhưng cách tính khác nhau – đúng nguyên lý polymorphism đã trình bày ở phần Estimator/Transformer.

Polymorphism với nhiều model

```
from abc import ABC, abstractmethod

class BaseModel(ABC):
    @abstractmethod
    def fit(self, X, y):
        ...
    @abstractmethod
    def predict(self, X):
        ...

class RuleBasedChurnModel(BaseModel):
    def fit(self, X, y):
        print("No training needed")
    def predict(self, X):
        return [1 if row["support_tickets"] >= 3 else 0 for row in X]

class AlwaysChurnModel(BaseModel):
    def fit(self, X, y):
        print("Always predicts churn")
    def predict(self, X):
        return [1 for _ in X]
```

Cùng giao diện fit/predict

```
X = [  
    {"support_tickets": 1},  
    {"support_tickets": 4}  
]  
  
models = [RuleBasedChurnModel(), AlwaysChurnModel()]  
  
for model in models:  
    model.fit(X, None)  
    print(model.predict(X))
```

Đây là polymorphism: cùng lời gọi method, mỗi object thực hiện theo cách riêng.

Mixin Pattern: kết hợp hành vi độc lập

```
class LoggingMixin:
    def log(self, message):
        print(f"[{self.__class__.__name__}] {message}")

class Trainer(LoggingMixin):
    def __init__(self, model):
        self.model = model

    def train(self, X, y, epochs):
        for epoch in range(epochs):
            self.log(f"epoch {epoch+1}/{epochs} - training...")

trainer = Trainer(model=RuleBasedChurnModel())
trainer.train(X, None, epochs=2)
```

Multiple inheritance qua Mixin cho phép tái sử dụng hành vi (logging, checkpointing) cho nhiều class Trainer khác nhau mà không cần lặp code.

Object State trong Data Science

Object state là trạng thái object lưu giữ sau khi được tạo hoặc sau khi gọi method.

- Scaler lưu trung bình và độ lệch chuẩn sau khi `fit()`.
- Model lưu hệ số sau khi huấn luyện.
- Dataset object lưu record và metadata.

Ví dụ Object State

```
class MeanScaler:
    def fit(self, values):
        self.mean_ = sum(values) / len(values)
        return self

    def transform(self, values):
        return [x - self.mean_ for x in values]
```

Sau khi `fit()`, object scaler đã lưu `mean_`.

Estimator, Transformer, Predictor

Vai trò	Method chính	Ý nghĩa
Estimator	<code>fit()</code>	Học từ dữ liệu
Transformer	<code>transform()</code>	Biến đổi dữ liệu
Predictor	<code>predict()</code>	Dự đoán kết quả
Model	<code>score()</code>	Đánh giá độ phù hợp

Quy ước API này được mô tả chi tiết trong Buitinck et al. (2013), *API Design for Machine Learning Software*, và là chuẩn thiết kế của Scikit-Learn.

Data Leakage

Data leakage xảy ra khi thông tin từ test data bị dùng trong quá trình huấn luyện hoặc tiền xử lý, dẫn tới đánh giá mô hình sai lệch theo hướng lạc quan (Kaufman et al., 2012).

Quy trình đúng: fit transformer trên train data, sau đó dùng transformer đã fit để transform cả train và test data. Không fit lại trên test data.

Pipeline như một object tổng hợp

```
class SimplePipeline:
    def __init__(self, transformer, model):
        self.transformer = transformer
        self.model = model

    def fit(self, X, y):
        X_new = self.transformer.fit_transform(X)
        self.model.fit(X_new, y)
        return self

    def predict(self, X):
        X_new = self.transformer.transform(X)
        return self.model.predict(X_new)
```

Pipeline là ví dụ điển hình của composition trong Data Science.

Vì sao PyTorch và OpenMMLab thiết kế theo OOP?

- Mọi layer, mọi mô hình trong PyTorch đều là một object kế thừa từ `nn.Module`.
- Một mô hình lớn (ResNet, Transformer) thực chất là **composition**: object chứa nhiều object con (layer) bên trong.
- `forward()` định nghĩa hành vi riêng của từng layer/model – đúng nguyên lý **polymorphism** đã học.
- OpenMMLab (MMDetection, MMClassification, MMEEngine...) mở rộng thêm **Registry Pattern** để xây dựng mô hình từ file config.

Thông điệp

Đọc hiểu OOP không chỉ để viết code – mà để đọc hiểu và tùy biến source code của PyTorch, OpenMMLab.

Mini-framework: Layer là một object

```
class Layer:
    def forward(self, x):
        raise NotImplementedError

    def __call__(self, x):
        return self.forward(x)

class Linear(Layer):
    def __init__(self, w, b):
        self.w, self.b = w, b
    def forward(self, x):
        return [self.w * xi + self.b for xi in x]

class ReLU(Layer):
    def forward(self, x):
        return [max(0, xi) for xi in x]
```

Minh họa nguyên lý OOP đằng sau `nn.Module`, không phải cài đặt tối ưu về mặt toán học (phép tính tensor thật sẽ học ở buổi NumPy/PyTorch).

Composition: ghép nhiều Layer thành một mô hình

```
class SimpleNN(Layer):
    def __init__(self, layers):
        self.layers = layers    # list các object Layer

    def forward(self, x):
        for layer in self.layers:
            x = layer(x)        # gọi __call__ -> forward của từng layer
        return x

model = SimpleNN([Linear(w=2.0, b=-1.0), ReLU()])
output = model([-2.0, 0.5, 3.0])
print(output)
```

SimpleNN không tự tính toán – nó ủy quyền (delegate) cho từng Layer con. Đây chính xác là cách `nn.Sequential` hoạt động trong PyTorch.

Đổi chiều với PyTorch thật

```
import torch.nn as nn

class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(in_features=10, out_features=32)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(in_features=32, out_features=1)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x

model = Net()
y = model(x)    # gọi __call__ của nn.Module -> tự động gọi forward(x)
```

So sánh với SimpleNN ở slide trước: `self.fc1`, `self.relu`, `self.fc2` là các object Layer được composition lại; `super().__init__()` chính là `super()` đã học ở phần Inheritance.

Custom Dataset trong PyTorch

```
import torch
from torch.utils.data import Dataset, DataLoader

class ChurnDataset(Dataset):
    def __init__(self, features, labels):
        self.features = features
        self.labels = labels

    def __len__(self):
        return len(self.features)

    def __getitem__(self, idx):
        x = torch.tensor(self.features[idx], dtype=torch.float32)
        y = torch.tensor(self.labels[idx], dtype=torch.float32)
        return x, y

dataset = ChurnDataset(features=[[32, 15.5, 2]], labels=[1])
loader = DataLoader(dataset, batch_size=32, shuffle=True)
```

`DataLoader` gọi `__len__` và `__getitem__` của `Dataset` – đúng hai magic method đã trình bày ở phần trước.

Registry Pattern trong OpenMMLab

```
MODELS = {}

def register_module(name):
    def wrapper(cls):
        MODELS[name] = cls
        return cls
    return wrapper

@register_module("resnet18")
class ResNet18:
    def build(self):
        print("Building ResNet18 backbone")

# Dang ky them backbone khac: @register_module("resnet50") ...

def build_model(cfg):
    return MODELS[cfg["type"]]() # tra cuu class trong dict roi khoi tao

cfg = {"type": "resnet18"}
model = build_model(cfg)
model.build()
```

Vì sao Registry Pattern hữu ích?

Đây chính là cơ chế đứng sau `@MODELS.register_module()`, `@BACKBONES.register_module()` trong `MMDetection`, `MMClassification`.

- Kiến trúc mô hình được mô tả bằng **config dict** (type: "resnet18"), không cần sửa code khi đổi backbone.
- Thêm mô hình mới chỉ cần viết class mới và gắn decorator đăng ký, không sửa code xây dựng (build) sẵn có.
- Registry kết hợp ba khái niệm đã học: **class**, **dict** và **decorator** (hàm bậc cao).

Xem thêm: Chen et al. (2019), *MMDetection: Open MMLab Detection Toolbox and Benchmark*, arXiv:1906.07155.

Tổng kết Module 3

- Built-in classes: `list`, `tuple`, `dict`, `set`, `str`.
- User-defined classes: attribute, method, `self`, `__init__`, class attribute.
- Encapsulation: `_protected`, `__private`, `@property`.
- `@classmethod` (alternative constructor), `@staticmethod` (hàm tiện ích).
- Magic methods: `__str__`, `__repr__`, `__add__`, `__len__`, `__getitem__`, `__call__`.
- Inheritance, `super()`, abstract base class (`abc`), polymorphism, Mixin.
- OOP trong Data Science: `DataFrame`, `Estimator/Transformer`, `Pipeline`.
- OOP trong Deep Learning: `nn.Module`, composition layer, `Dataset/DataLoader`, `Registry Pattern` (OpenMMLab).

Tài liệu tham khảo I

- Bolton, R. J., & Hand, D. J. (2002). Statistical fraud detection: A review. *Statistical Science*, 17(3), 235–249.
- Buitinck, L., et al. (2013). API design for machine learning software: Experiences from the scikit-learn project. *arXiv:1309.0238*.
- Chapman, P., et al. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS Inc.
- Chen, K., et al. (2019). MMDetection: Open MMLab detection toolbox and benchmark. *arXiv:1906.07155*.
- Conway, D. (2010). *The Data Science Venn Diagram*. drewconway.com.
- Davenport, T. H., & Patil, D. J. (2012). Data scientist: The sexiest job of the 21st century. *Harvard Business Review*, 90(10), 70–76.
- Dhar, V. (2013). Data science and prediction. *Communications of the ACM*, 56(12), 64–73.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- Gomez-Uribe, C. A., & Hunt, N. (2016). The Netflix recommender system: Algorithms, business value, and innovation. *ACM Transactions on Management Information Systems*, 6(4), 1–19.

Tài liệu tham khảo II

- Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.
- Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4), 1–21.
- Laney, D. (2001). *3D Data Management: Controlling Data Volume, Velocity, and Variety*. META Group Research Note.
- Linden, G., Smith, B., & York, J. (2003). Amazon.com recommendations: Item-to-item collaborative filtering. *IEEE Internet Computing*, 7(1), 76–80.
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 51–56.
- Obermeyer, Z., & Emanuel, E. J. (2016). Predicting the future — Big data, machine learning, and clinical medicine. *New England Journal of Medicine*, 375(13), 1216–1219.
- Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems (NeurIPS)*, 32.

- Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Reinsel, D., Gantz, J., & Rydning, J. (2018). *The Digitization of the World: From Edge to Core*. IDC White Paper.
- VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media.

Chuẩn bị cho buổi tiếp theo: NumPy Foundation

Thông điệp

OOP là cầu nối giữa Python cơ bản và cách các thư viện Data Science hiện đại được thiết kế.